

Day 63:- * Generator in python:-

- generator is one of the function
- The generator function always contains yield keyword.
- if the function contains return statement then it is called "Normal Function". Here return statement of function can return more number of values if required.
- if the function contains yield keyword then it is called "generator". Here yield statement returns the value only on demand and reduces the memory space.

Syntax). `def function_name (start, stop, step):`

yield value

- ⇒ The yield keyword is used for giving the value back to function call from function definition and continue the function execution until condition becomes false.
- ⇒ The Advantage of generator over functions concept is that it save lot of memory space in the case large sampling of data.
- ⇒ In other words function gives all the result at once and it take more memory space where as generators gives one value at a time when programmer requested and takes minimized memory space.

Non gen

```
r = range(10, 100, 5)
for val in r:
    print (val)
```

ex 1 # Generator

```
def KvrRange(val):
    i = 0
    while (i < val):
        yield i
        i = i + 1

# main program
g = KvrRange(5) # Here 'g' is an object of <class 'generator'>
print("Content of g=")
print(" — — — — — ")
print(next(g))
print(next(g))
print(next(g))
print(next(g))
print(next(g))
# print(next(g)) --- This stmt generator stop iteration
                        bcoz generator yielded all the demanded
                        values
```

ex(2).

```
def KvrRange(val):
    i = 0
    while (i < val):
        yield i
        i = i + 1

# main program
g = KvrRange(10)
print("Content of g=")
print(" — — — — — ")
while(5):
    try:
        print(next(g))
    except StopIteration:
        print(" — — — — — ")
        break
```

ex(3).

```
def KvrRange(Begin, End):
    while (Begin <= End):
        yield Begin
        Begin += 1 # Here += is called shorthand operator

# main program
g = KvrRange(10, 20)
# as it is like a list
```

```
g1 = KvrRange(100, 120)
# same as it is while ke baad same
```

ex(4) - Bas . def krrrange (Begin, end, step); - iske baad sab same -

ex(5) def getCourses():
 yield "PYTHON"
 yield "java"
 yield "osk"
 yield "Dsc"
 # main program
 cs = getCourses()
 print(next(cs)) =

* Iterators in python :-

• Why should we use iterators:-

→ In modern days, we have a lot of data in our hands, and handling this huge amount of data creates problems for everyone who wants to do some sort of analysis with that data. So, if you have ever struggled with handling huge amounts of data, and your machine running out of memory, then we use the concept of iterators in python.

→ Therefore, rather than putting all the data in the memory in one step, it would be better if we could work with it in bits or some small chunks, dealing with only that data that is required at that moment. As a result, this would reduce the load on our computer memory tremendously. and this is what exactly the iterators do.

→ Therefore you can use iterators to save a lot of memory, as iterators don't compute their items when they are generated, but only when they are called upon.

- ⇒ iterator in python is an object that is used to iterate over iterable objects like lists, tuples, dicts, str, and sets.
- ⇒ the iterator object is ~~made~~ initialized using the `iter()` function. it uses the `next()` function for iteration.
- ⇒ Here `iter()` is used for converting iterable object into iterator object.
- ⇒ `next()` is used for obtaining next elements of iterator object and if no next element then we get an exception called stopiteration.
- ⇒ on the object of iterator, we can't perform indexing and slicing operations because they supply the value on demand.

ex ①. `lst = [10, 20, 30, 40, 50, 60, 70, 80, 90]` # Here `lst` is one of the iterable object.

Convert iterable object into iterator object by using `iter(iterable object)`

```

lstiter = iter(lst)
print(" — — — — ")
print("Type of lst =", type(lst)) # <class, list>
print("Type of lstiter =", type(lstiter)) # <class, list_iterator>
print(" — — — — ")
print(next(lstiter))
print(next(lstiter))
print(next(lstiter))
while (True):
    by:
        print(next(lstiter))
    except stopiteration:
        print(" — — — — ")
        Break

```

ex(2). tpl = (10, "Rossum", 45.67, True, 2+3j)
 ex(3). set = {10, "Rossum", 45.67, True, 3+4j}
 ex(4). d = {10: "Python", 20: "DSC", 30: "Java", 40: "DSA", 50: "C++"}
 Ye dict_key iterator hai

isme Bay

```
try:
    k = next(diter)
    v = d[k] # or v = d.get(k)
    print("\t{} ==> {}".format(k, v))
```

ex(5). for k in diter:
 print("\t{} ==> {}".format(k, d.get(k)))
 print(" - - - ")
 print(" - - - ")
 OR → diter = iter(d) # rebuilding dict_key iterator
 for k in diter:
 print("\t{} ==> {}".format(k, d[k]))
 print(" - - - ")

ex(6). s = "PYTHON"
 → iske baad same rhege

```
for val in s:
    print("\t{}".format(val))
print(" - - - ")
```

ex(7). r = range(10, 100, 10).
 as it is

GENERATORS

```
In [1]: #non generators
r = range(10, 100, 5)
for val in r:
    print(val)
```

10
15
20
25
30
35
40
45
50
55
60
65
70
75
80
85
90
95

```
In [10]: #ex 1:-generators
def Arifrange(val):
    i=0
    while(i<val):
        yield i
        i=i+1

#main program
g=Arifrange(5)      #here 'g' is an object of <class, 'generator'>
print("content of g=",g,type(g))
print("=====")
print(next(g))
print(next(g))
print(next(g))
print(next(g))
print(next(g))
#print(next(g)) #this stmt generates stop iteration bcoz generator s yielded al
```

content of g= <generator object Arifrange at 0x0000029607F7A500> <class 'generator'>

```
=====
0
1
2
3
4
```

```
In [14]: #ex 2:-
def Arifrange(val):
    i=0
    while(i<=val):
        yield i
        i=i+1

#main program
g=Arifrange(10)
print("content of g=",g)
print("=====")
while(True):
    try:
        print(next(g))
```

```

except StopIteration:
    print("=====")
    break

```

content of g= <generator object Arifrange at 0x0000029606FA3A00>

```

=====
0
1
2
3
4
5
6
7
8
9
10
=====

```

```

In [29]: #ex 3:-
def Arifrange(Begin,End):
    while(Begin<=End):
        yield Begin
        Begin +=1          #here += is called short hand operators

#main program
g=Arifrange(10,20)
print("content of g=",g)
print("=====")
while(True):
    try:
        print(next(g))
    except StopIteration:
        print("=====")
        break
g1=Arifrange(100,150,)
while(True):
    try:
        print(next(g1))
    except StopIteration:
        print("=====")
        break

```

content of g= <generator object Arifrange at 0x0000029607F7AE00>

=====

10
11
12
13
14
15
16
17
18
19
20

=====

100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145

146
147
148
149
150

=====

```
In [27]: #ex 4:-
def Arifrange(Begin,End,step):
    while(Begin<=End):
        yield Begin
        Begin +=step
#main program
g=Arifrange(100,200,5)
while(True):
    try:
        print(next(g))
    except StopIteration:
        print("=====")
        break
```

100
105
110
115
120
125
130
135
140
145
150
155
160
165
170
175
180
185
190
195
200

=====

```
In [35]: #ex 5:-
def getcourses():
    yield "PYTHON"
    yield "JAVA"
    yield "DSA"
    yield "DSC"
#main program
crs=getcourses()
print(next(crs))
print(next(crs))
print(next(crs))
print(next(crs))
```

PYTHON
JAVA
DSA
DSC

iterators

```
In [39]: #ex 1:
lst=[10,20,30,40,50,60,70,80,90] #here lst is one of the iterable object
#convert iterable object into iterator object by using iter(iterable_object)
lstiter=iter(lst)
print("=====")
print("type of lst=",type(lst))
print("type of lstiter=",type(lstiter))
print("=====")
print(next(lstiter))
print(next(lstiter))
print(next(lstiter))
while (True):
    try:
        print(next(lstiter))
    except StopIteration:
        print("=====")
        break
```

```
=====
type of lst= <class 'list'>
type of lstiter= <class 'list_iterator'>
=====
10
20
30
40
50
60
70
80
90
=====
```

```
In [40]: #ex 2:-
tpl=(10,"rosum",45.67,True,2+3j)
#convert iterable object into iterator object by using iter(iterable_object)
tpliter=iter(tpl)
print("=====")
print("type of tpl=",type(tpl))
print("type of tpliter=",type(tpliter))
print("=====")
print(next(tpliter))
print(next(tpliter))
print(next(tpliter))
while (True):
    try:
        print(next(tpliter))
    except StopIteration:
        print("=====")
        break
```

```

=====
type of tpl= <class 'tuple'>
type of tpliter= <class 'tuple_iterator'>
=====
10
rosum
45.67
True
(2+3j)
=====

```

```

In [47]: #ex 3:-
d={10:"python",20:"dsc",30:"java",40:"dsa",50:"c++"}
diter=iter(d)
print("=====")
print("type of dict=",type(d))
print("type of dictiterator=",type(diter))
print("=====")
print(next(diter))
print(next(diter))
print(next(diter))
while (True):
    try:
        k=next(diter)
        v= d.get(k)
        print("\t{}----->{}".format(k,v))
    except StopIteration:
        print("=====")
        break

```

```

=====
type of dict= <class 'dict'>
type of dictiterator= <class 'dict_keyiterator'>
=====
10
20
30
40----->dsa
50----->c++
=====

```

```

In [54]: #ex 4:-
set=(10,"rosum",45.67,True,3+4j)
setiter=iter(set)
print("=====")
print("type of set=",type(set))
print("type of setiterator=",type(setiter))
print("=====")
print(next(setiter))
print(next(setiter))
print(next(setiter))
while (True):
    try:
        print(next(setiter))
    except StopIteration:
        print("=====")
        break

```

```
=====
type of set= <class 'tuple'>
type of setiterator= <class 'tuple_iterator'>
=====
10
rosum
45.67
True
(3+4j)
=====
```

In []:

In []: